

발표 대본 v24 (2026. 9. 30.)

- 대상 자료

- 1부: slides/pdf/osung_ai_digital_problem_solving_part1_workspace_v24_20260930.pdf (42쪽) - 2부: slides/pdf/osung_ai_digital_problem_solving_part2_ux_implementation_v24_20260930.pdf (35쪽) - 인쇄용 안내문: practice/osung_ai_digital_problem_solving_handout_a4_v24_20260930.pdf (A4 앞뒤)

- 작성 기준: SPEAKER_SCRIPT_GUIDE.md. 화면 글자를 그대로 읽지 않고, 화면에 없는 맥락·사례·주의점·전환을 말한다.

- 표기: [질문] 청중에게 묻기 · [예시] 현장 사례 · [시연] 화면 조작 · [주의] 오해하기 쉬운 점 · [전환] 다음 장 연결 · [선택 설명] 시간이 부족하면 줄이거나 생략

- 근거 번호 [n]은 docs/04_sources.md 번호다.

- 맨 끝에 '예상 질문과 답변'이 있다.

- 이전 대본: v1(55장)은 커밋 b057154, v21(1부 44쪽·2부 41쪽)은 커밋 1981b7b에 남아 있다.

준비

- 인쇄용 안내문을 참석자 수만큼 준비한다(양면, 긴 쪽 넘김). 앞면은 계획을 적는 면, 뒷면은 AI에게 할 말이다.

- 2부 20쪽 시연용으로 knollab-001 네 사이트를 브라우저 탭에 미리 열어 둔다.

- 실습 전에 사용할 웹 채팅 또는 전문 도구의 로그인·작업 폴더를 확인한다. 가능한 환경을 선택하며, 설치를 실습의 필수 조건으로 두지 않는다.

시간이 부족할 때

- 1부: 11·12쪽(화면·처리·기록, API와 MCP)은 핵심 한 줄만 말한다. 22쪽은 오른쪽 74% → 81%만 설명한다. 29쪽은 규칙·스킬·혹의 차이와 학교 사례만 말한다. 30쪽(에이전트)은 '기준과 결과물을 주어 새 대화에서 검토'만 남긴다.

- 2부: 18쪽 도구 표는 그림 시안이 필요할 때 참고할 첫 줄만 짚는다. 20쪽 knollab-001은 첫 화면 비교만 보여 주고 시연은 줄인다. 실습(25~32쪽) 시간과 마지막 공유 시간은 줄이지 않는다.

1부. 문제 정의와 AI 작업환경

1부 01. 표지

안녕하세요. 오늘은 두 부로 나눠 진행합니다. 1부에서는 무엇을 만들지 정하고, AI가 우리 기준으로 일하게 하는 방법을 봅니다. 2부에서는 학생이 헤매지 않는 화면을 설계하고 직접 만들어 봅니다.

코딩을 배우는 시간은 아닙니다. 선생님들이 이미 가진 학생 이해를 AI가 쓸 수 있는 형태로 바꾸는 시간입니다. 나눠 드린 안내문은 앞면이 계획을 적는 면, 뒷면이 AI에게 할 말입니다. 강의 중간중간 앞면을 채워 가겠습니다.

[질문] AI로 수업 자료나 작은 프로그램을 직접 만들어 보신 분이 계신가요? (손을 들게 해 경험 수준을 파악한다)

1부 02. 1부에서 함께 볼 다섯 가지

다섯 묶음 중 앞의 두 개는 '무엇을 만들지', 뒤의 세 개는 'AI에게 어떻게 맡길지'에 관한 이야기입니다. 도구 이름이 여러 개 나오지만 외우실 필요는 없습니다. 1부가 끝났을 때 문제를 한 문장으로 쓸 수 있고, AI에게 줄 파일 몇 개가 무엇인지 알면 충분합니다.

[전환] 먼저 무엇을 만들지부터 정하겠습니다.

1부 03. 01 작은 불편에서 시작한다

첫 번째 묶음입니다. '어떤 앱을 만들까'보다 '어떤 불편을 없앨까'를 먼저 정합니다.

1부 04. 검색해서 따라 하던 일을 이제는 도구로 만든다

예전에는 컴퓨터를 한 시간 뒤에 끄고 싶으면 명령어를 검색해서 복사했습니다. 윈도우에서는 시간을 초로 바꿔 3600을 넣어야 하고, 틀리면 다시 검색해야 했죠. 지금은 필요한 것을 말하면 AI가 버튼 몇 개짜리 프로그램을 만들어 줍니다. 이렇게 코드를 직접 쓰지 않고 말로 설명해 AI가 만들게 하는 방식을 흔히 '바이브코딩'이라고 부릅니다.

[주의] 코드를 전혀 몰라도 된다는 뜻은 아닙니다. 무엇이 필요한지 분명히 말하는 능력이 코드를 쓰는 능력만큼, 어쩌면 그보다 더 중요해졌다는 뜻입니다.

[시연] (선택) 미리 만들어 둔 작은 도구(예: 수업 타이머)를 잠깐 보여 준다. 예약 종료 프로그램은 실제로 컴퓨터가 꺼질 수 있으니 시연하지 않는다.

1부 05. 학교에는 이런 작은 불편이 많다

여섯 칸을 모두 읽지는 않겠습니다. 공통점은 '매번', '다시', '또'입니다. 반복해서 하는 설명, 학생마다 바꿔 만드는 자료가 좋은 출발점입니다.

[예시] 매일 아침 일정 카드를 다시 붙이는 일, 급식 순서를 매번 말로 설명하는 일 같은 것입니다.

[질문] 이번 주에 '또 이걸 하네' 싶었던 일이 있으셨나요? 한 가지만 메모해 두세요. 조금 뒤 안내문 앞면 2번에 편하게 적고, 2부 실습 주제로 씁니다.

1부 06. 좋은 문제는 작고, 분명하고, 확인할 수 있다

떠올린 불편을 다섯 가지 기준으로 걸러 봅니다. 가장 중요한 것은 마지막, '잘됐는지 확인할 수 있다'입니다. '생활 전반을 관리하는 앱'은 언제 완성인지 알 수 없습니다. '하차 벨 누르기 연습'은 학생이 처음부터 끝까지 혼자 해 보면 완성입니다.

[주의] 처음 만드는 분일수록 문제를 크게 잡습니다. 작게 잡아야 오늘 안에 끝까지 가 볼 수 있습니다.

1부 07. 문제를 한 문장으로 적어 본다

이 네 칸은 생각을 정리할 때 참고하는 틀입니다. 떠오르는 내용부터 짧게 적거나, 한 문장으로 이어 적으셔도 됩니다. 아직 모르는 것은 비워 두고 AI와 이야기하거나 만들어 보면서 보완하겠습니다. 특히 '무엇을 어려워하는지'는 학생을 아는 선생님의 관찰이 중요한 부분입니다. 안내문 앞면 2번의 자유 공간에 메모합니다. 이 표의 모든 칸을 먼저 채울 필요는 없습니다.

[예시] '주문 순서를 어려워한다'와 '메뉴판 글자를 읽기 어려워한다'는 전혀 다른 자료로 이어집니다. 앞은 단계 연습 자료가 되고, 뒤는 그림으로 고르는 자료가 됩니다.

1부 08. 무엇을 만들지는 사용 조건이 정한다

문제가 같아도 쓰는 조건에 따라 만드는 방법이 달라집니다. 오른쪽 예시 질문에서 필요한 것만 골라 생각하고, 안내문 앞면 2번에 사용 상황을 함께 메모하시면 됩니다. 학교에서는 두 가지가 특히 중요합니다. 학교 PC는 프로그램 설치가 막혀 있는 경우가 많다는 것, 그리고 학생 이름과 사진 같은 개인정보입니다.

[주의] 개인정보는 '놓지 않는다'로 시작하는 것이 가장 안전합니다. 꼭 넣어야 한다면 먼저 학교의 개인정보 담당자와 상의합니다.

1부 09. 조건이 바뀌면 만드는 방식이 바뀐다

조건을 채우면 만드는 방식이 거의 정해집니다. 학생이 태블릿으로 바로 여는 자료는 웹 페이지, 곧 웹앱이면 됩니다. 나이스처럼 인터넷 창에서 주소로 여는 프로그램이 웹앱입니다. 설치할 필요가 없고, 고치면 모든 기기에 바로 반영됩니다.

예약 종료처럼 컴퓨터 전원을 다뤄야 하면 한글 프로그램처럼 컴퓨터에 설치하는 프로그램이 필요합니다. 웹 페이지는 보안 때문에 컴퓨터를 끄거나 파일을 마음대로 건드릴 수 없기 때문입니다. 여러 선생님이 같은 기록을 보려면 저장 공간과 로그인이 필요하고, 그때부터 개인정보 검토가 따라옵니다.

그래서 오늘 첫 도구는 저장과 로그인이 없는 웹 페이지로 만듭니다. 끝까지 가 보기가 가장 쉽고, 학생용 연습 자료는 대부분 이것으로 충분합니다.

[전환] 방금 웹 페이지, 설치형, 로그인 같은 말이 나왔습니다. 다음 묶음에서 이런 말을 알아듣는 정도만 정리하겠습니다.

1부 10. 02 AI와 말이 통하는 최소 지식

개발 용어를 배우는 시간이 아닙니다. AI가 "웹앱으로 할까요, 설치형으로 할까요?"라고 물었을 때 당황하지 않을 정도만 봅니다.

1부 11. 보이는 화면, 보이지 않는 처리, 남는 기록

[선택 설명] 식당으로 비유하면, 손님이 보는 홀이 프론트엔드, 보이지 않는 주방이 백엔드, 재료를 쌓아 두는 창고가 데이터베이스입니다. 오늘 만들 자료는 '홀만 있는 식당'으로 충분합니다. 버튼을 누르면 다음 화면이 나오는 정도는 화면만으로 만들 수 있습니다.

기록이 꼭 필요하다면 먼저 '브라우저 저장'을 씁니다. 그 태블릿의 그 브라우저에만 남는 방식입니다.

[주의] 다른 기기에서는 보이지 않고, 공용 태블릿이라면 다음 학생에게도 보입니다. 학생 이름은 저장하지 않습니다.

1부 12. API와 MCP: 다른 프로그램과 연결하는 약속

[선택 설명] API는 프로그램끼리 정보를 주고받는 창구입니다. MCP는 AI가 다른 도구를 쓸 수 있게 하는 표준 연결 방법입니다. 공식 소개에서도 'AI용 USB-C 포트'에 비유합니다. 연결 방식을 한 번 맞춰 두면 여러 AI가 같은 도구를 쓸 수 있다는 뜻입니다. [7]

[주의] 연결은 곧 권한입니다. GitHub에 연결된 AI는 파일을 지울 수도 있고, 메일에 연결된 AI는 메일을 보낼 수도 있습니다. 필요한 것만 연결합니다.

1부 13. GitHub와 배포

GitHub는 파일을 보관하는 온라인 저장소인데, 가장 큰 장점은 '되돌리기'입니다. AI가 고친 결과가 마음에 들지 않으면 어제 버전으로 돌아갈 수 있습니다. 배포는 다른 사람이 주소로 들어올 수 있게 공개하는 일입니다. GitHub Pages를 쓰면 무료로 주소를 만들 수 있고, 그 주소를 QR코드로 만들면 학생이 태블릿으로 바로 엽니다. [16]

[주의] 무료 요금제에서는 공개 저장소를 써야 하고, 만든 사이트는 누구나 볼 수 있습니다. 학생 이름, 사진, 기록은 넣지 않습니다.

1부 14. 개발 지식은 AI의 질문에 답할 때 쓴다

이 네 질문은 AI가 실제로 자주 묻는 질문입니다. 오른쪽 대답에는 개발 용어가 거의 없습니다. "태블릿에서 바로 열어야 해요", "저장 안 해도 돼요"처럼 교실 조건을 말하면 됩니다.

[주의] AI가 묻지 않고 바로 만들기 시작하면 조건을 스스로 정해 버립니다. 로그인 화면이 붙어 있거나 설치가 필요한 결과가 나오는 이유입니다. 그래서 조건을 먼저 주거나, 먼저 질문하게 요청합니다. 이 요청문은 39쪽에서 봅니다.

1부 15. 무엇으로 만들지 모르면 조건을 주고 비교를 부탁한다

무엇으로 만들지 모를 때는 조건과 비교 기준을 함께 주고 비교를 부탁합니다. 핵심은 비교 기준입니다. 기준 없이 "뭐가 좋아?"라고 물으면 가장 흔한 답이 돌아옵니다. 난이도, 비용, 고치기 쉬움, 개인정보 위험처럼 우리에게 중요한 기준을 주면 우리 상황에 맞는 답이 돌아옵니다.

추천은 받되, 결정은 선생님이 합니다.

[전환] 여기까지가 무엇을 만들지 정하는 이야기였습니다. 이제 이 일을 AI에게 맡길 때 가장 자주 부딪히는 문제로 넘어갑니다.

1부 16. 03 질문 한 번에서 작업환경으로

한 번 묻고 답을 받는 사용법에서, AI가 며칠 동안 같은 기준으로 일하게 하는 사용법으로 넘어갑니다.

1부 17. 대화가 길어지면 AI는 기준을 놓친다

[질문] AI와 긴 작업을 해 보신 분, 이런 경험 있으시죠? (왼쪽 목록에서 한두 개만 짚는다)

이유는 단순합니다. AI는 현재 전달된 지침·자료·대화 범위에서 작업합니다. 이 범위를 '컨텍스트'라고 부릅니다. 대화가 길어지면 앞부분의 기준을 놓칠 수 있고, 새 대화에 이전 작업의 세부사항이 모두 전달되는 것은 아닙니다.

[주의] 요즘 도구에는 기억 기능도 있지만, 무엇을 기억할지 우리가 정확히 정할 수 없고 도구마다 다릅니다. 중요한 기준은 직접 파일로 남기고, AI가 실제로 읽도록 전달하는 편이 확실합니다.

1부 18. 좋은 질문 한 번보다 좋은 작업환경이 오래 간다

해결 방법은 설명을 대화가 아니라 파일에 남기는 것입니다. 파일은 새 대화에서도, 다른 AI 도구에서도, 다른 선생님이 열어도 그대로 있습니다.

[예시] 교생 선생님이 올 때마다 학급 규칙을 처음부터 말로 설명하는 대신, 학급 운영 안내 한 장을 건네는 것과 같습니다.

1부 19. 하네스: AI가 믿을 만하게 일하도록 갖춘 환경

이렇게 AI가 일하는 주변 환경을 갖추는 일을 요즘 '하네스'라고 부릅니다. 말에 채우는 마구에서 온 말입니다. 말은 힘이 세지만, 마구가 있어야 그 힘을 원하는 방향으로 씁니다.

OpenAI의 한 팀은 사람이 코드를 한 줄도 직접 쓰지 않고 AI 에이전트만으로 제품을 만들어 봤습니다. 그 뒤 사람의 일이 코드를 쓰는 것에서 세 가지로 바뀌었다고 정리했습니다. 환경을 갖추고, 의도를 분명히 적고, 확인하는 구조를 만드는 것입니다. [1]

여기서 에이전트는 질문에 답만 하는 AI가 아니라, 스스로 파일을 읽고 도구를 쓰며 일을 진행하는 AI입니다. Codex, Claude Code, Antigravity 같은 도구가 여기에 속합니다.

1부 20. 결과가 나쁠 때 '더 잘해'라고 하지 않는다

같은 글에서 가장 와닿는 대목입니다. 일이 잘 안 될 때 해결책은 거의 '더 열심히'가 아니었다고 합니다. 대신 '무엇이 빠져 있어서 못 했나?'를 묻고, 그것을 AI가 볼 수 있는 곳에 채워 넣었습니다. [1]

[예시] 버튼이 계속 작게 나온다면, 버튼 크기 기준이 어디에도 적혀 있지 않았다는 뜻입니다. 기준 파일에 한 줄을 적으면 다음 화면부터 해결됩니다.

[전환] 이런 파일을 잘 갖춘 공개 사례를 하나 보겠습니다.

1부 21. 해커톤 우승자가 공개한 것은 '작업환경'이었다

Everything Claude Code, 줄여서 ECC라고 부르는 공개 자료입니다. 2025년 뉴욕에서 열린 Anthropic과 Forum Ventures의 해커톤 우승자가 만들었고, 해커톤 때부터 쓰던 설정을 다듬어 2026년 1월 GitHub에 공개했습니다. 지금은 별표가 26만 개가 넘을 만큼 많이 참고됩니다. [9]

눈여겨볼 점은 공개한 것이 '잘 쓴 프롬프트 모음'이 아니라 규칙, 스킬, 자동 점검, 도구 연결을 묶은 '작업환경'이라는 것입니다.

[주의] 공식 표준이 아니라 개인이 공개한 자료이고, 설정이 수백 개에 이릅니다. 통째로 설치하면 AI가 읽을 것만 많아집니다. 오늘은 구조만 참고하고, 33쪽에서 최소 구성을 따로 보여 드립니다.

1부 22. 문제를 가장 잘 아는 사람이 도구를 만들었다

2026년 Anthropic의 'Built with Opus 4.7' 해커톤 수상자들입니다. 왼쪽을 보시면 의사 출신 개발자, 전자제품 수리 기사, 대학에서 컴퓨터과학을 가르치는 교육자, 그리고 코딩 경험이 없는 스무 살 청년입니다. 공통점은 모두 자기가 매일 겪는 문제를 풀었다는 것입니다. 교육자 수상자는 "학생들이 AI로 코드를 만들지만 자기 코드를 이해하지 못한다"는 수업 고민에서 출발했습니다. 선생님들과 가장 가까운 고민입니다. [10]

오른쪽이 오늘 강의와 가장 가까운 사례입니다. 특별상을 받은 청년은 목수인 아버지를 오래 인터뷰해서 진단 규칙, 현장에서 쓰는 말, 기준 가격, 흔한 실수를 파일 하나로 정리했습니다. AI에게 주는 기본 지시문은 바꾸지 않고 이 파일만 더했는데, 숙련 목수의 판단과 맞는 정도가 74%에서 81%로 올랐습니다. [10]

선생님들 머릿속에 있는 학생 이해도 이런 파일이 될 수 있습니다. "선택지가 셋을 넘으면 멈춘다", "글보다 사진에 먼저 반응한다" 같은 것들입니다.

[주의] 학생 이해를 파일로 쓸 때는 이름 대신 '학생 A'처럼 쓰고, 수행 특성만 적습니다.

[전환] 그럼 이런 작업환경은 무엇으로 이루어져 있는지 보겠습니다.

1부 23. 04 작업환경을 이루는 것들

용어가 몇 개 나옵니다. 하나씩 외우기보다 먼저 전체 지도를 보고, 하나씩 실제 파일로 짚어 가겠습니다.

1부 24. 작업환경은 학교 업무 체계와 닮았다

작업환경을 이루는 여섯 가지를 학교 업무에 빗대 보면 익숙합니다. 규정이 있고, 업무 편람이 있고, 매뉴얼과 담당자, 교무 시스템, 결재와 인수인계가 있습니다. 오른쪽 '실제 모습'을 보시면 대부분 글자로 된 파일입니다. 메모장으로도 만들 수 있습니다.

[주의] 비유는 이해를 돕는 장치일 뿐 딱 들어맞지는 않습니다. 다음 장부터 실제 파일을 보면서 정확한 의미를 붙입니다.

[전환] 앞으로 나오는 예시는 모두 '카페 주문 연습' 자료 하나로 이어집니다.

1부 25. 규칙 파일: 매번 말하지 않아도 지킬 것

규칙 파일은 AI가 일을 시작하기 전에 매번 읽는 파일이고, 이름은 보통 AGENTS.md입니다. '.md'는 마크다운이라는 간단한 글 형식인데, 메모장에 쓴 글이라고 생각하시면 됩니다.

요령은 짧게 쓰는 것입니다. Claude Code 안내서는 파일이 길수록 AI가 덜 지킨다고 설명하고, OpenAI도 새 모델이 나올 때마다 규칙을 다시 보고 필요 없어진 줄은 지우라고 권합니다. [2][5]

[예시] 같은 부탁을 두 번 하게 되면 그때가 한 줄 더할 때입니다. "버튼 크게 해 줘"를 두 번 말했다면 '버튼 높이 64px 이상'을 적어 둡니다.

1부 26. 도구 이름은 달라도 같은 파일을 쓸 수 있다

도구마다 규칙 파일 이름과 스킴 폴더가 조금씩 다릅니다. 반가운 점은 점점 같은 이름으로 모이고 있다는 것입니다. Codex와 Antigravity는 AGENTS.md를 읽고, Claude Code도 CLAUDE.md가 없으면 AGENTS.md를 읽습니다. [3][5][6]

그래서 AGENTS.md를 기준으로 두면, 오늘 Codex로 만들다가 내일 Claude Code로 이어서 해도 같은 규칙이 적용됩니다.

[주의] 이 표는 2026년 9월 공식 문서 기준입니다. 자주 바뀌는 부분이니 쓰실 때 도구 안내를 한 번 확인하세요. 스킴 부르는 기호(\$, /)는 도구마다 다르다는 것만 기억하시면 됩니다.

1부 27. 프로젝트 문서: AI가 알아야 할 배경

규칙이 '지켜라'라면 프로젝트 문서는 '알아 뉘라'입니다. 누구를 위한 자료인지, 어떤 행동을 연습하는지, 이번에 어디까지 만드는지를 적습니다. 1부 앞에서 쓴 문제 한 문장과 사용 조건이 그대로 이 파일이 됩니다.

가장 효과가 큰 줄은 마지막 '하지 않는 것'입니다. 적어 두지 않으면 AI는 친절하게 결제 화면, 로그인, 기록 저장까지 붙여 줍니다.

[전환] 화면 기준은 DESIGN.md에 따로 둡니다. 2부에서 자세히 봅니다.

1부 28. 스킬: 자주 하는 일을 매뉴얼로 만든다

스킬은 자주 하는 일의 매뉴얼입니다. 규칙과 다른 점은 읽는 시점입니다. 규칙은 매번 전부 읽고, 스킬은 평소에는 이름과 한 줄 설명만 보다가 그 일이 필요할 때 전체를 읽습니다. 그래서 스킬이 여러 개여도 AI가 덜 헷갈립니다. [4][6]

[예시] 오른쪽은 '화면 점검' 스킬입니다. 자료를 만들 때마다 "한 화면에 행동이 하나인지, 버튼이 큰지 봐 줘"라고 말하는 대신 이 스킬을 부르면 늘 같은 순서로 점검합니다.

[주의] AI는 한 줄 설명(description)을 보고 스킬을 고릅니다. 언제 쓰는 스킬인지 짧고 분명하게 적습니다. [2]

1부 29. 규칙은 매번, 스킬은 필요할 때, 혹은 정해진 순간에

AI 도구 안내를 찾아보시면 커맨드, 워크플로, 혹은 같은 말이 계속 나옵니다. 처음에 가장 헷갈리는 부분이라 한 장으로 정리했습니다. 구분하는 기준은 두 가지입니다. 언제 쓰이는지, 그리고 누가 부르는지입니다.

규칙 파일은 도구가 정한 위치와 형식에 두면 일을 시작할 때 읽힙니다. 브라우저 채팅에서는 지침에 넣거나 함께 전달합니다. 스킬은 그 일이 필요할 때만 읽는데, AI가 설명을 보고 스스로 고르기도 하고 사람이 이름으로 부르기도 합니다. 혹은 조금 다릅니다. 파일을 고친 직후처럼 정해진 순간이 되면 반드시 실행되고, AI가 할지 말지 고르지 않습니다. 사람이 지나가면 저절로 꺼지는 센서등과 비슷합니다. 그래서 '고친 뒤에는 꼭 검사한다'처럼 빠지면 안 되는 일에 씁니다. [24]

[선택 설명] 헷갈리는 이유는 '워크플로'라는 말이 두 가지로 쓰이기 때문입니다. 하나는 기획, 구현, 검토처럼 일을 하는 순서라는 보통 뜻입니다. 다른 하나는 도구의 기능 이름입니다. 자주 쓰는 요청문을 저장해 두고 /이름으로 부르는 기능인데, 도구마다 이름이 달랐습니다. Antigravity는 워크플로, Claude Code는 커맨드, Codex는 커스텀 프롬프트라고 불렀습니다.

이 기능들이 최근 스킬로 모이고 있습니다. Codex는 커스텀 프롬프트를 더 이상 권하지 않고 스킬을 쓰라고 안내합니다. Claude Code는 예전 커맨드 파일도 아직 동작하지만 예전 형식이라고 안내하고, 스킬도 똑같이 /이름으로 부릅니다. Gemini의 Gem도 개인 계정에서는 2026년 11월부터 스킬로 바뀝니다. [20][22]

스킬로 모이는 까닭은 할 수 있는 일이 더 많기 때문입니다. 저장된 요청문은 사람이 불러야만 쓰였고, Codex 커스텀 프롬프트처럼 내 컴퓨터에만 저장되어 다른 사람과 나누기 어려운 것도 있었습니다. 스킬은 AI가 필요할 때 스스로 꺼내 쓰고, 순서뿐 아니라 양식과 참고 자료까지 폴더째 담고, 프로젝트 폴더에 넣어 함께 쓸 수 있습니다.

그래서 이렇게 기억하시면 됩니다. 워크플로는 일의 순서이고, 그 순서를 담아 두는 곳이 스킬입니다.

[주의] 오늘 만들 작은 자료에는 혹은 필요 없습니다. 인터넷에서 예전 방식의 '커맨드'나 '워크플로' 만드는 법을 보시면, 지금은 같은 내용을 스킬로 만들면 된다고 생각하시면 됩니다.

[선택 설명] 일의 순서를 프로그램으로 아예 고정해 두고 AI가 그 길로만 가게 만드는 방법도 있습니다. 결과를 예측하기 쉽지만 상황이 바뀌면 길을 바꾸지 못합니다. 오늘 쓰는 에이전트는 반대로 스킬을 참고하면서 상황에 맞게 스스로 판단합니다. [22]

[전환] 스킬이 일하는 방법이라면, 다음은 그 방법으로 일하는 담당자 이야기입니다.

1부 30. 에이전트: 일이 커지면 역할을 나눠 맡긴다

[선택 설명] 일이 커지면 역할을 나눕니다. 기획, 화면 설계, 구현, 검토를 서로 다른 대화나 다른 에이전트에게 맡기는 방식입니다. 학교의 업무 분장과 같습니다.

다만 오늘 만들 작은 자료는 AI 하나로 충분합니다. 하나만 권한다면 검토를 새 대화에 맡기는 것입니다. 같은 대화 안에서는 자기가 내린 결정을 전제로 보기 쉬워서, 새 대화에 DESIGN.md와 최신 HTML 파일을 함께 첨부

하고 "이 기준으로 검토해 줘"라고 요청합니다. 화면 캡처는 배치 확인을 돕고, 실제 동작은 직접 눌러 확인합니다. 이 이야기는 2부 23쪽에서 다시 합니다.

1부 31. 도구 연결: AI가 직접 열어 보고 확인하게 한다

앞에서 본 MCP 이야기입니다. 파일, GitHub, 브라우저, Figma를 연결하면 AI가 말로만 답하지 않고 직접 열어 보고 확인합니다. 그중에서도 브라우저 연결이 중요합니다. AI가 만든 화면을 스스로 열어 눌러 보고 고칠 수 있기 때문입니다.

ChatGPT의 '앱', Claude의 '커넥터'도 같은 역할을 합니다. 이름이 달라도 '외부 도구를 AI에게 연결한다'는 뜻입니다. [8]

[주의] 학교 업무 계정을 연결할 때는 특히 신중해야 합니다. 개인 계정과 학교 계정을 섞지 말고, 필요한 권한만 허용합니다.

1부 32. 완료 기준을 먼저 주면 AI가 스스로 확인하고 고친다

AI에게 일을 맡길 때 가장 중요한 한 줄은 완료 기준입니다. "만들어 줘"가 아니라 "이렇게 되면 완료야"를 먼저 줍니다. 그러면 AI가 만들고, 실행해 보고, 기준과 비교하고, 틀린 곳을 고치는 반복을 스스로 돌립니다. 이런 반복을 '루프'라고도 부릅니다. OpenAI도 최신 모델에는 시작하기 전에 완료를 정의해 주라고 권합니다. [2]

[주의] 반복에는 멈출 기준도 필요합니다. 같은 실패가 세 번 반복되면 멈추고 사람에게 묻게 하는 것이 현장에서 쓰기 좋은 규칙입니다. 그리고 AI가 "완료했습니다"라고 하면 "무엇으로 확인했어?"라고 물어보세요.

1부 33. 처음에는 이만큼만 갖추면 된다

처음에는 폴더 하나에 파일 네 개면 충분합니다. 29쪽 표의 '처음에는' 칸과 같은 이야기이고, 안내문 앞면 5번에도 이 네 파일이 있습니다. 스킬은 같은 일을 여러 번 하게 될 때 만들면 됩니다.

ECC처럼 수백 개의 설정을 갖추 필요는 없습니다. 쓰지 않는 설정이 많으면 AI가 읽을 것만 늘어납니다. 문제가 생길 때마다 한 줄씩 늘려 가는 것이 가장 오래 가는 방법입니다.

[전환] 그럼 이 폴더를 어디에 만들고, 긴 작업은 어떻게 이어 가는지 보겠습니다.

1부 34. 05 작업공간과 인계

마지막 묶음입니다. 폴더를 만들고, 대화가 길어졌을 때 이어 가는 방법입니다.

1부 35. 프로젝트마다 폴더 하나를 만든다

프로젝트마다 폴더를 하나씩, 문서 폴더처럼 찾기 쉬운 곳에 만듭니다. 헛갈리기 쉬운 것이 오른쪽 설정 폴더입니다. AI 도구를 설치하면 사용자 폴더에 점(.)으로 시작하는 숨김 폴더가 생기는데, 도구의 설정 공간이라 직접 만질 일이 거의 없습니다.

AI 도구에서는 '폴더 열기'로 프로젝트 폴더 자체를 엽니다. 그래야 AI가 그 폴더의 AGENTS.md를 읽습니다.

[예시] 나중에 GitHub에 올릴 자료라면 폴더 이름을 영어(예: cafe-order)로 해 두면 편합니다. GitHub 저장소 이름에는 한글을 쓸 수 없습니다.

[시연] (선택) AI 도구에서 '폴더 열기'를 한 번 보여 준다.

1부 36. 공통 규칙과 이 프로젝트의 규칙을 나눈다

규칙에도 층이 있습니다. 모든 프로젝트에 적용할 내 취향은 공통 규칙에, 이 자료에만 해당하는 내용은 프로젝트 폴더의 규칙에 둡니다. 학교 규칙 아래 학급 규칙이 있는 것과 같습니다.

Codex는 공통 규칙을 먼저 읽고, 그다음 프로젝트 폴더의 규칙을 읽습니다. 작업 폴더에 가까운 규칙일수록 나중에 읽어서 더 구체적인 지시가 됩니다. [3]

[주의] 학생 특성이나 화면 기준을 공통 규칙에 넣으면 다른 프로젝트에도 섞여 들어갑니다. 프로젝트 내용은 프로젝트 폴더에 둡니다.

1부 37. 대화가 길어지면 인계 파일을 쓰고 새 대화로 넘긴다

대화가 길어지면 AI가 앞 내용을 흐리게 기억한다고 말씀드렸습니다. 그래서 오른쪽 순서처럼 기능 하나를 끝낼 때마다 인계 파일을 쓰게 하고, 새 대화에서는 그 파일부터 읽게 합니다. 담당 업무를 넘길 때 인수인계서를 쓰는 것과 같습니다.

왼쪽이 실제 인계 파일입니다. 지금 상태, 정한 것, 안 된 방법, 다음 할 일, 네 줄이면 충분합니다. 가장 중요한 줄은 '안 된 방법'입니다. 예를 들어 태블릿 브라우저는 소리가 저절로 재생되지 않게 막아 두는 경우가 많아서, 버튼을 누르면 소리가 나게 바꿨습니다. 이걸 적어 두지 않으면 새 대화의 AI가 같은 방법을 또 시도하느라 시간을 씁니다.

인계 파일은 직접 쓰지 않아도 됩니다. "지금까지 한 것을 HANDOFF.md로 정리해 줘"라고 하면 AI가 씁니다. 선생님은 읽어 보고 틀린 곳만 고치면 됩니다.

[주의] 대화 전체를 복사해 넘기지 않습니다. 필요한 상태만 짧게 넘기는 것이 핵심입니다.

1부 38. 이 강의자료도 같은 방식으로 만들었다

오늘 보시는 슬라이드도 이 방식으로 만들었습니다. GitHub의 OSUNGSCHOOL 저장소에 작업 규칙, 화면 기준, 문구 기준, 대본 기준을 파일로 두고, AI가 그 파일을 읽고 슬라이드를 만들었습니다.

만드는 동안 같은 문제가 반복되면 슬라이드만 고치지 않고 기준 파일을 고쳤습니다. 용어만 늘어놓은 슬라이드가 계속 나오자 '용어를 목록으로 늘어놓지 않는다'는 규칙을 넣었고, 같은 내용이 여러 장에 반복되자 '한 흐름은 한 장에'라는 규칙을 더했습니다. AI에게 한 번 더 고쳐 달라고 하는 것보다, 규칙 한 줄이 다음 작업을 계속 바꿉니다.

[시연] (선택) github.com/LUCKYBRIDGE/OSUNGSCHOOL에서 AGENTS.md와 DESIGN.md를 열어 보여 준다.

1부 39. 처음 요청할 때 쓰는 문장

첫 요청문은 자유 메모로 시작합니다. 진행에 꼭 필요한 질문만 짧게 받고, 정하지 않은 것은 제안이나 미정으로 남기게 합니다. 개요와 규칙은 짧은 초안부터 만들고 제작 중 보완합니다. 웹 채팅에는 그 내용을 전달하고, 전문 도구에는 작업 폴더에 저장하도록 요청합니다. Codex는 AGENTS.md, Claude Code는 CLAUDE.md를 쓸 수 있고 기존 규칙은 먼저 읽게 합니다. [3][5]

괄호 부분에는 떠오른 아이디어나 사용 상황을 편하게 넣습니다. 네 칸과 사용 조건 표는 메모를 돕는 참고 자료입니다. 안내문 뒷면 '요청 1'이 이 문장이고, 2부 실습에서 그대로 씁니다.

1부 40. 이어서 작업할 때 쓰는 문장

다음 날 이어서 할 때 쓰는 문장입니다. 작업 규칙·인계·최신 결과물을 먼저 읽게 하고, 한 번에 하나만 진행합니다. 완료 기준과 확인 결과·미확인을 남기고 인계를 갱신합니다. 웹 채팅에는 파일을 첨부하고 전문 도구에는 같은 폴더에서 읽게 하면 됩니다. 안내문 뒷면 '요청 5'입니다.

1부 41. 1부에서 가져갈 세 가지

1부를 정리하면 이렇습니다. 문제와 사용 상황을 메모하고 필요한 정보를 보완하며, 설명은 파일로 남기고, 완료는 확인한 결과로 판단합니다.

쉬는 시간 전에 하나만 부탁드립니다. 5쪽에서 떠올린 불편을 안내문 앞면 2번에 편하게 메모해 주세요. 7쪽의 질문은 생각이 막힐 때 참고하고, 아직 모르는 것은 미정으로 남겨도 됩니다.

[전환] 2부에서는 이 위에 화면을 올립니다. 기능이 돌아가는 것과 학생이 실제로 쓸 수 있는 것은 다르다는 이야기부터 시작합니다.

1부 42. 참고 자료

오늘 나온 도구 정보는 2026년 9월 공식 문서로 확인했습니다. 기능과 요금은 자주 바뀌니 실제로 쓰실 때 한번 더 확인해 주세요. 전체 목록은 저장소의 docs/04_sources.md에 있습니다.

2부. UX/UI 설계와 구현·검증 실습

2부 01. 표지

2부입니다. 1부에서 무엇을 만들지 정하고 AI가 기준을 지키게 하는 방법을 봤다면, 2부에서는 학생이 실제로 쓸 수 있는 화면을 설계하고 직접 만들어 봅니다. 뒷부분은 실습입니다.

2부 02. 2부에서 함께 할 여섯 가지

앞의 다섯 묶음은 실습에서 그대로 따라갈 순서이기도 합니다. 흐름을 설계하고, 기준을 파일로 남기고, 필요하면 그림으로 먼저 확인하고, 작게 만들어 확인한 뒤 공개합니다.

2부 03. 01 잘 돌아가는 것과 잘 쓰이는 것은 다르다

AI 덕분에 '돌아가는 것'은 쉬워졌습니다. 이제 어려운 것은 '잘 쓰이는 것'입니다.

2부 04. AI는 그럴싸한 화면을 빠르게 만든다

AI에게 화면을 부탁하면 대개 보기 좋은 결과가 나옵니다. 문제는 보기 좋은 화면과 우리 학생이 쓸 수 있는 화면이 다르다는 점입니다. AI는 기능을 더하는 쪽으로 친절합니다. 무엇을 빼야 하는지 아는 사람은 학생을 아는 선생님입니다.

판단 기준은 하나면 충분합니다. 우리 학생이 혼자 처음부터 끝까지 할 수 있는가입니다.

2부 05. 예: "영화 예매 사이트처럼 만들어 줘"의 첫 결과

영화 예매 연습 자료를 '실제 예매 사이트처럼' 만들어 달라고 하면 이런 결과가 나오기 쉽습니다. 실제 사이트를 흉내 냈기 때문에 영화도 많고, 좌석도 많고, 광고까지 뜹니다.

[질문] 우리 반 학생이 이 화면을 처음 본다면 어디부터 눌러야 할지 알 수 있을까요?

현실적인 것은 좋지만, 현실을 그대로 옮기면 연습해야 할 행동이 문혀 버립니다.

2부 06. 교사가 다듬으면: 연습할 행동이 보이는 화면

같은 영화 예매를 선생님이 다듬으면 이렇게 됩니다. 한 화면에 한 단계만 있고, 위에 지금 몇 단계인지 보이고, 버튼은 크고, 언제든 이전으로 돌아갈 수 있습니다.

[주의] 무조건 줄이자는 이야기가 아닙니다. 학습 목표에 맞게 단계를 설계하는 것입니다. 학생이 익숙해지면 영화를 네 편으로 늘리고 좌석 선택을 더해 실제에 가깝게 바꿔 갈 수 있습니다. 언제 무엇을 늘릴지 정하는 판단이 곧 도메인 지식입니다.

2부 07. UX는 사용하는 과정, UI는 보이는 화면

UI는 눈에 보이는 버튼, 글자, 색입니다. UX는 학생이 시작해서 끝낼 때까지 겪는 전체 과정입니다. 학교 행사로 치면 UI는 안내 표지판이고 UX는 동선입니다.

표지판을 아무리 예쁘게 만들어도 동선이 꼬이면 길을 잃습니다. 그래서 흐름을 먼저 정하고 화면은 나중에 정합니다.

2부 08. 화면보다 흐름을 먼저 정한다

카페 주문 화면의 흐름을 다섯 단계로 나눠 본 예시입니다. 단계 수는 사용자와 목적에 맞게 바꿀 수 있습니다. 아래 따는 잘못 눌렀을 때 선택을 바꾸는 길입니다. 특수교육 자료에서 가장 자주 빠지는 것이 바로 이 되돌아가는 길입니다.

[예시] 실제 키오스크 앞에서 학생이 가장 당황하는 순간도 잘못 눌렀을 때입니다. 연습 자료에서는 '잘못 눌러도 괜찮다'는 경험을 먼저 주는 것이 중요합니다.

흐름은 그림으로 그려도 좋지만, AI에게는 번호를 매긴 글로 주는 것이 가장 정확합니다. "1. 시작: 큰 시작 버튼 하나, 2. 메뉴 고르기: 음료 그림 2개 중 하나"처럼 쓰면 화면이 몇 개인지, 화면마다 버튼이 몇 개인지가 정해집니다. 원하는 음료나 크기를 고르는 활동에는 하나의 정답이 없습니다. 자유 선택을 오답으로 처리하지 않습니다. '큰 컵을 고르세요'처럼 정답이 있는 과제에서만 힌트나 정답 강조를 씁니다. 어떤 도움을 언제 줄지는 학생과 목표를 아는 선생님이 정합니다.

흐름이 떠오르면 안내문 앞면 2번의 자유 공간에 그려 보세요. 정해진 화면 수에 맞출 필요는 없습니다.

[전환] 그림 우리 학생을 위한 화면에는 어떤 원칙이 있을까요?

2부 09. 02 우리 학생을 위한 화면 원칙

화면 원칙을 보겠습니다. 원칙은 출발점이고, 최종 기준은 우리 반 학생입니다.

2부 10. 특수교육 자료의 화면 원칙

여섯 가지를 모두 설명하지는 않겠습니다. 이 가운데 두 가지는 웹 접근성 국제 기준(WCAG 2.2)에도 있습니다. 누르는 대상의 크기와, 색만으로 뜻을 전하지 않는 것입니다. 이 기준은 누르는 대상을 최소 24픽셀, 더 높은 수준에서는 44픽셀 이상으로 정합니다. 우리 학생들에게는 이보다 크게, 64픽셀 정도에서 시작하시길 권합니다.

[17]

[질문] 여섯 가지 중에서 우리 반 학생에게 가장 중요한 두 가지를 골라 보세요. 실습 때 DESIGN.md에 가장 먼저 적을 내용입니다.

2부 11. 같은 자료도 학생에 따라 조절할 수 있게 만든다

같은 자료를 학생마다 새로 만들 필요는 없습니다. 처음부터 '교사용 설정'을 넣어 달라고 하면 선택지 수, 소리 안내, 가격 계산 같은 것을 켜고 끌 수 있습니다. 같은 주소에서 설정만 바꿉니다.

[주의] 기능을 빼는 것이 아니라 학생의 수행 수준과 목표에 맞추는 것입니다. 설정 화면은 학생이 실수로 열지 않도록 구석에 작게 두거나, 길게 눌러야 열리게 해 달라고 요청합니다.

2부 12. 정답이 있는 과제에서는 도움을 조절한다

이번 장은 정답이 있는 학습 과제의 예입니다. 자유 선택에는 정답·오답 판정을 넣지 않습니다. 학습 과제에서 틀렸을 때 무엇을 보여 주느냐가 학습을 좌우합니다. 처음에는 다시 기회를 주고, 또 틀리면 힌트를 주고, 그래도 어려우면 정답을 보여 주고 함께 누릅니다. 특수교육에서 말하는 최소-최대 촉진처럼 도움을 조금씩 늘리는 방식입니다.

맞았을 때는 바로, 구체적으로 알려 줍니다. "잘했어요"보다 "큰 컵을 골랐어요"가 무엇을 잘했는지 알려 줍니다.

[주의] 학생에 따라서는 처음부터 정답을 보여 주고 도움을 점점 줄이는 방식이 맞을 수도 있습니다. 어떤 방식을 쓸지는 선생님이 정하고, 그 규칙을 흐름에 적어 AI에게 줍니다.

2부 13. 03 DESIGN.md: 화면 기준을 파일로

이제 이런 화면 기준을 AI가 잊지 않도록 파일로 남기는 방법입니다.

2부 14. DESIGN.md는 AI를 위한 화면 기준서다

1부에서 규칙 파일을 봤습니다. DESIGN.md는 그중 화면 기준만 따로 모은 파일입니다. 색, 글자 크기, 버튼, 간격, 그리고 하지 않을 것을 적고, AI는 화면을 만들 때마다 이 파일을 먼저 읽습니다.

Google Stitch가 쓰던 이 형식을 2026년 4월 누구나 쓸 수 있게 공개했고, 지금은 여러 AI 도구가 이 파일을 읽을 수 있습니다. 파일 맨 위(왼쪽 칸)에는 색 코드와 글자 크기처럼 정확한 값을, 그 아래(오른쪽 칸)에는 왜 그렇게 쓰는지와 무엇을 피하는지를 적습니다. [12][13]

복잡해 보이지만 직접 쓰실 필요는 없습니다. 초안은 AI가 쓰고, 선생님은 버튼 높이나 하지 않을 것처럼 학생과 관련된 줄을 고칩니다. 오늘 보시는 슬라이드의 화면 기준도 같은 형식의 DESIGN.md로 관리하고 있습니다.

[주의] 형식은 아직 초기 단계(alpha)입니다. 형식을 완벽하게 맞추는 것보다 우리 학생에게 맞는 기준을 적는 것이 더 중요합니다.

2부 15. DESIGN.md를 만드는 요청문

마지막 줄을 꼭 넣으세요. "내가 고칠 만한 부분 3가지를 알려 줘." 이렇게 하면 AI가 확신이 없는 부분을 먼저 알려 주고, 선생님의 판단이 들어갈 자리가 생깁니다. 안내문 뒷면 '요청 2'가 이 문장입니다.

[전환] 기준이 생겼으면 코드를 만들기 전에 그림으로 먼저 확인해 볼 수 있습니다.

2부 16. 04 코드보다 먼저 그림으로 확인한다

코드보다 그림이 싸다는 이야기입니다. 도구가 여러 개 나오지만 필요한 것만 고르면 됩니다.

2부 17. 코드로 고치기 전에 그림으로 실패해 본다

화면이 중요한 자료라면 코드를 만들기 전에 그림으로 먼저 실패해 봅니다. 그림 시안은 마음에 들지 않으면 버리고 다시 그리면 됩니다. 코드는 여러 번 고칠수록 꼬이기 쉽습니다.

핵심은 두 번째 단계, 학생의 눈으로 보는 것입니다. 처음 보는 학생이 무엇을 눌러야 할지 바로 알 수 있는지, 그것 하나만 봐도 됩니다. 마지막 단계에서 정한 시안과 DESIGN.md를 함께 넘기는 것은 어떤 도구를 쓰든 같습니다.

[주의] 간단한 자료라면 이 단계는 건너뛰어도 됩니다. 필수가 아니라 선택입니다.

2부 18. 시안 도구는 필요한 만큼만 고른다

모양만 빨리 보고 싶으면 그림 생성이 가능한 도구에 시안을 부탁하면 됩니다. 전문 도구로 구현할 때는 그 시안을 작업 폴더에 두고 읽게 할 수 있습니다. 시안은 선택입니다. 나머지 세 줄은 이런 도구가 있다는 정도만 알고 가시면 됩니다.

[선택 설명] Google Stitch는 말로 화면을 만들고 여러 안을 나란히 비교할 수 있습니다. 화면을 이어 실제처럼 눌러 보는 시안을 만들고, DESIGN.md를 가져오거나 내보내며, Antigravity 같은 개발 도구로 넘길 수 있습니다. [11][12]

[선택 설명] Claude Design은 대화하면서 다듬고, 원하는 부분에 댓글을 달거나 직접 고칠 수 있습니다. 디자인 시스템을 적용하고 PDF·PPTX·HTML로 내보낼 수 있으며, 완성되면 Claude Code로 넘겨 구현합니다. 2026년 4월 유료 플랜 사용자 대상 연구 프리뷰로 나왔습니다. [14]

[선택 설명] Figma는 디자이너들이 많이 쓰는 설계 도구입니다. 반복되는 버튼을 한 번에 고치는 컴포넌트, 내용에 맞춰 크기가 바뀌는 오토 레이아웃, 색과 간격을 이름으로 관리하는 변수가 강점입니다. Figma MCP로 AI와 연결하면 AI가 화면 그림을 보고 추측하는 대신 디자인 구조를 그대로 읽습니다. 지원되는 환경에서는 AI가 Figma에 직접 화면을 그리거나, 만든 웹 화면을 Figma로 가져와 다듬을 수도 있습니다. 여러 선생님이 함께 자료를 관리할 때 검토해 볼 만합니다. [15]

어떤 도구를 쓰든 기준은 DESIGN.md 하나입니다. 오늘 그림 시안으로 시작해 나중에 Figma로 바꿔도 기준 파일은 그대로 씁니다.

[주의] Stitch·Claude Design·Figma는 반드시 써야 하는 도구가 아닙니다. 기능과 요금, 권한 조건이 자주 바뀌니 쓰기 전에 확인하세요.

2부 19. 그림 시안은 이렇게 요청하고 이렇게 고친다

DESIGN.md를 함께 주고, 화면 비율과 단계를 분명히 말합니다. 첫 그림을 정답으로 받아들이지 말고, 아래 예시처럼 짧게 고칠 점을 말해 두세 번 다시 그림니다. 안내문 뒷면 '요청 3'입니다.

[주의] 그림 속 한글은 틀리게 나오는 경우가 있습니다. 그림 시안에서는 글자보다 배치와 크기를 봅니다. 정확한 글자는 구현할 때 들어갑니다.

[시연] (선택) ChatGPT나 Gemini로 '메뉴 고르기' 시안을 한 장 만들어 보여 준다.

[전환] 기준 파일과 Figma가 결과를 실제로 얼마나 바꾸는지, 제가 직접 해 본 실험을 보겠습니다.

2부 20. 같은 목표를 네 가지 작업환경에서 만들어 봤다

오늘 연수를 준비하면서 직접 해 본 비교 실험입니다. 특수학교 학생이 카페에서 음료를 주문하는 과정을 연습하는 웹사이트를 같은 목표로 네 번 만들었습니다. 바꾼 것은 AI에게 준 기준과 자료입니다. [23]

- A는 요청문만 주고 AI에게 맡겼습니다.
- B는 여기에 DESIGN.md를 더했습니다. 정답 화면을 정해 주지 않고, 인지 부담·선택 표시·바로 주는 피드백·쉬운 복구·터치 크기처럼 판단 기준만 적은 파일입니다.
- C는 B와 글자 하나까지 같은 DESIGN.md에 더해, 미리 고정해 둔 Figma 설계를 Figma MCP로 읽게 했습니다.
- D는 Figma 설계를 규격으로 삼아 적극 활용하는 방식이고, 지금 작업 중입니다.

A·B·C는 같은 단계별 요청문으로 버전 1, 2, 3을 만들었습니다. 각각 다른 ChatGPT 프로젝트에서 '프로젝트 전용 메모리'로 진행해서, 다른 대화에서 쌓인 기억이 섞이지 않게 했습니다. 조건을 똑같이 맞추려고 기억까지 나눈 것입니다.

첫 화면만 봐도 다릅니다. A와 B는 설명과 그림이 많고, C는 단계를 카드로 정리했고, D는 시작 버튼 하나에 집중합니다. D는 작업 중이라 바뀔 수 있습니다.

[시연] 네 사이트를 나란히 띄우고 같은 단계(음료 고르기 → 온도·크기 고르기)를 차례로 눌러 본다. 선택했을 때의 표시, 잘못 눌렀을 때 돌아가는 길, 버전이 올라가도 모양이 유지되는지를 비교한다. 주소:

luckybridge.github.io/knollab-001-a-ai-only/, .../knollab-001-b-design-md/,
.../knollab-001-c-design-figma-mcp/, .../knollab-001-d-design-figma-mcp-actively/

[주의] 한 번 해 본 실험이라 어느 방식이 항상 낫다고 일반화하지 않습니다. 같은 조건에서도 AI의 결과는 매번 조금씩 달라집니다. 차이는 강사가 실제로 눌러 보며 확인한 것만 말합니다.

[전환] 이제 실제로 만들고 확인하는 단계입니다.

2부 21. 05 만들고, 확인하고, 공개한다

요령은 세 가지입니다. 작게 만들고, 직접 확인하고, 주소 하나로 엮는다.

2부 22. 한 번에 한 화면씩 요청하고 확인한다

AI에게 전체를 한 번에 맡기면 많은 것이 한꺼번에 만들어져서 어디가 틀렸는지 찾기 어렵습니다. 위 순서처럼 첫 화면 하나를 만들고 확인한 뒤 다음 화면을 붙입니다.

아래가 첫 화면을 요청하는 문장입니다. 따를 문서, 만들 범위, 완료 기준이 들어 있습니다. 1부 40쪽의 '이어서 작업할 때 쓰는 문장'과 같은 구조입니다.

안내문 뒷면 요청 4는 두 환경에서 공통으로 씁니다. 웹 채팅에는 기준과 최신 결과물을 첨부하고, 만든 파일을 내려받아 확인합니다. 작은 웹 화면이라면 모양과 동작을 포함한 HTML 파일 하나로 달라고 해도 됩니다. 미리 보기가 제공되면 그 안에서 눌러 볼 수 있습니다.

Codex나 Claude Code에서는 작업 폴더의 규칙·개요·설계 기준과 기존 파일을 읽고 필요한 파일을 만들거나 고치게 합니다. 실행·검사가 가능하면 수행하고 방법과 결과를 남기게 합니다. 브라우저 확인은 도구와 권한이 있을 때 가능하므로, 해 보지 못한 것은 미확인으로 남기고 선생님이 직접 확인합니다.

요청 하나에 화면 하나, 완료 기준 하나. 이 원칙만 지켜도 실패가 크게 줄어듭니다.

[시연] (선택) 채팅 AI에 이 요청문을 넣고 미리보기로 첫 화면을 눌러 본다.

2부 23. 검토는 읽어 보고, 검증은 직접 해 본다

검토는 읽어 보고 판단하는 것이고, 검증은 직접 해 보고 확인하는 것입니다. 행사 공문으로 치면 검토는 결재자가 문서를 읽어 보는 것이고, 검증은 그날 행사장이 정말 예약돼 있는지 전화해 보는 것입니다.

검토는 새 대화에 맡깁니다. 같은 대화 안에서는 AI가 자기가 내린 결정을 전제로 보기 쉬워서, 새 대화에 DESIGN.md와 최신 HTML 파일을 함께 첨부하고 "이 기준으로 검토해 줘"라고 요청합니다. 화면 캡처는 배치 확인을 돕고, 실제 동작은 직접 눌러 확인합니다.

검증의 마지막은 선생님이 직접 합니다. 오른쪽 다섯 가지, 5분이면 됩니다. 경험상 문제가 가장 많이 보이는 것은 선택을 바꾸거나 잘못 누른 뒤 돌아가 보는 것과 실제 사용할 기기에서 열어 보는 것입니다. 컴퓨터에서는 멀쩡하던 버튼이 태블릿에서는 너무 붙어 있는 경우가 많습니다. 마지막 줄은 글을 읽기 어려운 학생을 위한 확인입니다. 글자를 손으로 가리고도 그림만으로 끝까지 갈 수 있는지 봅니다.

AI가 "완료했습니다"라고 해도 그것은 검증이 아닙니다. 안내문 앞면 4번에서 결과와 미확인을 남깁니다. 웹 화면에는 이 동작 확인을 적용하고, 안내문·포스터에는 내용과 읽기 쉬움을 확인합니다.

2부 24. 공개하고, 학생이 써 보고, 멈춘 곳부터 고친다

왼쪽이 공개하는 순서입니다. 프로젝트 폴더를 GitHub 저장소로 올리고, 웹 화면은 index.html로 저장해 저장소 맨 위에 올립니다. Pages 메뉴에서 main 브랜치와 /(root)를 게시 위치로 지정하고, 배포가 끝난 뒤 실제 주소를 열어 확인합니다. 올리는 일도 AI에게 부탁할 수 있고, GitHub 웹 화면에 파일을 끌어다 놓아도 됩니다. 저장소 이름은 영어로 정합니다. 한글은 쓸 수 없고, 저장소 이름이 그대로 주소 끝에 붙습니다. 주소를 QR코드로 만들어 두면 학생이 태블릿 카메라로 바로 엮는다. [16]

[주의] 무료 요금제에서는 공개 저장소를 쓰고, 사이트는 누구나 볼 수 있습니다. 학교에서 지킬 것은 27쪽에서 한 번에 정리합니다.

오른쪽이 공개한 뒤입니다. 학생이 실제로 써 보면 만든 사람이 예상하지 못한 곳에서 멈춥니다. 그때 바로 고치지 말고 아래 예처럼 먼저 적습니다. 어느 화면에서, 무엇을 하려다, 무엇을 했는지입니다. 그리고 기능을 더하기

전에 멈춘 곳부터 고칩니다. 이 기록을 AI에게 그대로 주면 무엇을 고쳐야 할지 정확히 압니다. 고친 뒤 다시 올리면 같은 주소에 바로 반영됩니다.

[시연] (선택) 미리 만든 저장소에서 Pages를 켜는 화면을 보여 준다.

[전환] 이제 직접 해 보겠습니다.

2부 25. 06 실습: 수업·생활·업무 자료 만들기

실습입니다. 오늘 목표는 멋진 자료가 아니라, 작은 자료 하나를 처음부터 끝까지 완성해 보는 경험입니다.

2부 26. 쓸 수 있는 작업환경을 골라 시작한다

오늘 실습은 웹 채팅과 전문 도구 중 사용할 수 있는 환경을 고릅니다. 이미 Claude Code나 Codex를 쓰는 선생님은 그 작업 폴더에서 진행하고, 웹 채팅을 쓰는 선생님은 새 대화나 프로젝트·Gem에서 시작합니다. 준비가 안 된 도구를 새로 설치해야 하는 활동은 아닙니다.

웹 채팅에서는 메모와 기준, 최신 파일을 붙이거나 첨부합니다. 프로젝트·Gem의 지침과 파일을 활용할 수도 있습니다. 가능한 미리보기 또는 내려받은 결과 파일을 직접 확인합니다. 메뉴와 기능은 계정·기기·설정에 따라 다를 수 있습니다. [18][19][20][21]

전문 도구에서는 기존 파일과 규칙을 먼저 읽고, 같은 폴더에 결과물을 만들거나 고치도록 요청합니다. Codex의 AGENTS.md와 Claude Code의 CLAUDE.md처럼 도구가 읽는 규칙 파일을 씁니다. 기존 AGENTS.md를 공유한다면 CLAUDE.md에서 @AGENTS.md로 참조할 수 있습니다. 파일마다 같은 규칙을 반복해서 적을 필요는 없습니다. [3][5]

공통 흐름은 메모, 필요한 기준 정리, 작은 결과 만들기, 직접 확인입니다. 달라지는 것은 기준을 전달하고 결과를 저장·실행하는 방법입니다. 전문 도구의 실행·검사도 사용 가능한 도구와 권한에 따라 달라집니다. 확인하지 못한 것은 미확인으로 남기고, 최종 결과는 선생님이 실제 기기에서 확인합니다.

2부 27. 학교에서 쓸 때 지킬 여섯 가지

실습 전에 학교에서 쓸 때 지킬 것을 한 번에 정리합니다. 그중 두 가지를 특히 강조하겠습니다.

첫째, 개인정보는 넣지 않습니다. 이름, 사진, 진단명 대신 '학생 A'처럼 쓰고, 수행 특성만 적습니다. 채팅 AI에 넣은 내용은 서비스 회사의 서버로 전송되고 저장될 수 있습니다. 대화 내용을 모델 개선에 쓸지 고르는 설정을 두는 서비스도 있으니 설정 메뉴에서 확인하세요. [27]

둘째, AI는 교사가 쓰고, 학생은 완성된 화면만 씁니다. 서비스마다 나이 기준이 있습니다. ChatGPT는 13세 이상이고 18세 미만은 보호자 동의가 필요합니다. Gemini Gem은 개인 계정의 국가별 나이 기준을 따르며 한국에서는 14세 이상입니다. 학교 계정은 관리자 설정과 별도 조건을 확인합니다. Claude는 18세 이상만 쓸 수 있습니다. 오늘 요청문대로 만든 연습 화면은 AI와 연결되지 않은 보통 웹 페이지라, 학생이 놀러도 AI와 대화하지 않습니다. [20][25][26]

나머지는 앞에서 이미 말씀드린 것입니다. 무료 공개 주소는 누구나 볼 수 있고, 그림은 직접 만들었거나 사용 허락이 분명한 것만 씁니다. 학교 계정은 필요한 것만 연결하고, 안내 문구나 가격 같은 내용은 AI가 틀릴 수 있으니 선생님이 확인합니다.

[주의] 학교와 교육청의 AI 활용 지침이 있으면 그것을 먼저 따릅니다. 안내문 뒷면에도 같은 목록이 있습니다.

2부 28. 실습 주제: 해결해 보고 싶은 작은 일 하나

수업·생활·업무에서 해결해 보고 싶은 작은 일을 고릅니다. 생활 연습 화면, 수업 타이머, 읽기 쉬운 안전 안내문 모두 좋습니다. 3~5단계는 생활 연습 화면을 작게 시작하는 예시일 뿐입니다. 안내문 앞면 2번에 이미 쓴 메모가 있으면 그대로 쓰세요.

[질문] 옆 선생님과 1분만 이야기해 보세요. 어떤 장면을 고르셨나요?

2부 29. 오늘의 목표: 작게, 끝까지

실습에서 가장 흔한 실패는 목표가 너무 큰 것입니다. 로그인, 관리자 화면, 기록, 통계를 한꺼번에 넣으면 핵심 화면도 완성하기 어렵습니다. 예를 들어 음료를 고르고 확인하는 작은 화면부터 시작할 수 있습니다. 주제에 따라 한 장짜리 안내문이나 타이머 기능 하나여도 좋습니다.

결과물을 열어 의도한 내용이나 동작을 확인하고, 잘된 점·고칠 점·미확인을 남기면 됩니다. 결과물과 개요·설계 기준 등 필요한 기록을 함께 남깁니다. 웹 화면은 만들면서 규칙과 인계 문서도 보완합니다. 모든 예시 항목을 채우는 것이 성공 기준은 아닙니다. 웹 채팅에서는 기준·개요·인계와 결과 파일을 받아 함께 저장합니다. 전문 도구에서는 이 기록과 결과물이 같은 작업 폴더에 남았는지 확인합니다. 그러면 다음 주에 다시 열어도 AI가 바로 이어서 일할 수 있고, 다른 선생님이 이 폴더를 받아 자기 반에 맞게 고쳐 쓸 수도 있습니다.

2부 30. 실습 순서

막히면 기본 흐름을 참고합니다. 시간과 순서는 예시입니다. 이미 생각한 부분부터 시작하고, 미정은 만들면서 보완합니다.

[주의] 시간이 모자라면 3단계의 그림 시안은 건너뛰고, 마지막 공유 시간은 꼭 남겨 둡니다. 다른 선생님의 자료를 보는 것이 가장 큰 배움이 됩니다.

[시연] 1단계를 시작할 때 1부 39쪽 요청문을 다시 띄워 둔다. 안내문 뒷면 '요청 1'과 같다.

2부 31. 생각이 막히면 참고할 세 가지 질문

이 세 질문은 생각이 막힐 때 참고하는 질문입니다. 안내문 앞면 2번의 넓은 메모 공간에 떠오르는 것부터 적고, 이미 앞에 쓴 내용은 다시 쓰지 않아도 됩니다. 한 공간에 메모로 정리하셔도 좋습니다. 아직 정하지 못한 것은 '미정'으로 남기고, 이 메모를 '요청 1'과 함께 AI에게 주어 PROJECT_OVERVIEW.md 초안을 정리하게 합니다.

만들기 전에 '우선 무엇을 해 보고, 어떻게 확인할지'는 함께 정해 보세요. 큰 계획을 완성하는 대신 작은 목표부터 시작하고, 해 본 결과에 따라 바뀌어도 됩니다.

2부 32. 막혔을 때는 이렇게 말한다

오류가 나면 오류 메시지를 그대로 붙여 넣습니다. 상황을 설명하려고 애쓰는 것보다 메시지를 그대로 주는 편이 훨씬 빠릅니다.

AI가 너무 많이 바꿨을 때는 되돌리게 하고, 대화가 길어지면 인계 파일을 쓰게 한 뒤 새 대화로 옮깁니다. 같은 실패가 세 번이면 멈추고 요청을 더 작게 나눕니다. 이 표는 안내문 뒷면에도 있습니다.

막히면 편하게 손을 들어 주세요.

2부 33. 오늘 기억할 네 문장

오늘 내용을 네 문장으로 정리했습니다. 이 네 문장은 어떤 도구를 쓰든 그대로 적용됩니다. 도구는 계속 바뀌지만, 누구에게 무엇이 왜 필요한지 아는 사람은 바뀌지 않습니다. 그 사람이 바로 선생님들입니다.

2부 34. 참고 자료

2부에서 나온 도구 정보의 출처입니다. 1부와 마찬가지로 2026년 9월 기준이니 쓰실 때 한 번 더 확인해 주세요.

2부 35. 질문과 나눔

질문을 받겠습니다. 그리고 부탁이 하나 있습니다. 오늘 만든 자료를 다음 주 수업에서 학생과 한 번만 써 보세요. 학생이 멈춘 곳을 적어 두시면 그 기록이 다음 버전의 가장 좋은 설계 자료가 됩니다.

(자주 나올 질문과 답은 아래 '예상 질문과 답변'에 있다.)

예상 질문과 답변

- '알아 둘 것'은 답하기 전에 강사가 알고 있어야 할 사실이고, '이렇게 답한다'는 말로 할 답이다.
- 요금·나이 기준·기능은 바뀔 수 있으니 발표 직전에 다시 확인한다. 근거 번호는 docs/04_sources.md를 따른다.

비용과 도구

Q1. 돈이 드나요? 무료로 할 수 있나요?

- 알아 둘 것: ChatGPT 프로젝트는 무료 요금제에서도 쓸 수 있다 [18]. Claude도 무료 요금제에서 대화 옆 화면(아티팩트)을 만든다 [21]. 무료 요금제는 사용량 제한이 있다. Claude Design은 유료 플랜 대상 연구 프리뷰다 [14]. GitHub Pages는 공개 저장소라면 무료다 [16]. 설치해서 쓰는 도구는 도구마다 요금·사용량 조건이 다르다.
- 이렇게 답한다: "오늘 실습은 무료 계정으로도 할 수 있게 짰습니다. 다만 무료는 사용량 제한이 있어서 긴 작업은 중간에 멈출 수 있습니다. 그럴 때는 HANDOFF.md로 정리해 두고 다음에 이어 가시면 됩니다. 설치해서 쓰는 도구는 요금 조건이 도구마다 달라서, 쓰시기 전에 확인하시는 게 좋습니다."

Q2. ChatGPT, Gemini, Claude 중 무엇이 가장 좋나요?

- 알아 둘 것: 도구의 우열을 단정하지 않는다(AGENTS.md 6절). 웹 채팅의 지침·파일과 가능한 미리보기를 활용할 수 있다 [18][19][20][21]. 전문 도구에서는 작업 폴더의 기준과 파일로 진행한다 [3][5]. 규칙과 개요는 글이라 다른 도구로 옮겨 쓸 수 있다.
- 이렇게 답한다: "지금 쓰고 계신 것을 쓰시면 됩니다. 웹 채팅으로도 할 수 있고, 이미 Claude Code나 Codex를 쓰면 그 작업 폴더에서 하셔도 됩니다. 규칙과 개요는 그냥 글이라서 나중에 다른 도구로 옮겨도 그대로 씁니다. 도구를 바꾸는 것보다 우리 학생 기준을 잘 적는 것이 결과를 더 크게 바꿉니다."

Q3. 학교 PC에 Codex나 Claude Code를 설치해도 되나요?

- 알아 둘 것: 학교 PC는 설치가 막혀 있는 경우가 많다. 설치형 에이전트는 폴더 속 파일을 직접 읽고 고치며 명령을 실행할 수 있다.
- 이렇게 답한다: "학교 PC 설치하는 먼저 학교 정보 담당 선생님과 상의해 주세요. 오늘은 사용 가능한 환경을 고르면 됩니다. 설치가 어렵다면 웹 채팅으로, 이미 전문 도구를 쓰는 개인 PC라면 작업 폴더에서 진행하세요. 설치형 도구는 파일을 직접 고치기 때문에 중요한 파일이 없는 작업 폴더에서 쓰시는 게 안전합니다."

Q4. knollab-001 실험에서는 어느 방식이 가장 좋았나요?

- 알아 둘 것: 한 번 해 본 실험이고, D는 아직 작업 중이다. 같은 조건에서도 AI 결과는 매번 조금씩 달라진다 [23].
- 이렇게 답한다: "한 번 해 본 실험이라 어느 방식이 늘 낫다고 말씀드리기는 어렵습니다. 어디가 다른지는 직접 눌러 보며 함께 보시죠. 저는 선택했을 때의 표시와 잘못 눌렀을 때 되돌아가는 길을 먼저 봅니다. 네 사이트 주소는 저장소에 있으니 나중에 직접 비교해 보셔도 됩니다."

개인정보와 학생 사용

Q5. 학생 정보를 넣어도 되나요?

- 알아 둘 것: 채팅 AI에 넣은 내용은 서비스 회사의 서버로 전송되고 저장될 수 있다. 대화를 모델 개선에 쓰지 고르는 설정을 두는 서비스도 있다 [27]. 학교·교육청의 개인정보 지침이 우선한다.

- 이렇게 답한다: "넣지 않는 것을 기본으로 합니다. 이름·사진·진단명 대신 '학생 A'처럼 쓰고, '선택지가 셋을 넘으면 멈춘다'처럼 수행 특성만 적어도 AI가 충분히 좋은 자료를 만듭니다. 꼭 필요한 경우에는 학교 개인정보 담당자와 먼저 상의해 주세요."

Q6. 학생이 직접 AI를 써도 되나요?

- 알아 둘 것: ChatGPT는 13세 이상이고 18세 미만은 보호자 동의가 필요하다 [25]. Gem은 개인 계정 기준 한국 14세 이상이며, 학교 계정은 별도 조건을 확인한다 [20]. Claude는 18세 이상만 쓸 수 있다 [26]. 오늘 요청문대로 만든 화면은 AI를 부르지 않는 보통 웹 페이지다.
- 이렇게 답한다: "서비스마다 나이 기준이 있어서, 오늘 방식은 선생님이 AI로 자료를 만들고 학생은 완성된 화면만 쓰는 것입니다. 오늘 만드는 연습 화면은 AI와 연결되지 않은 보통 웹 페이지라서, 학생이 눌러도 AI와 대화하지 않습니다."

Q7. 공개 주소로 올리면 누가 볼 수 있나요?

- 알아 둘 것: 무료 GitHub Pages는 공개 저장소에서 쓰고, 사이트는 인터넷에 공개된다 [16].
- 이렇게 답한다: "주소를 아는 사람은 누구나 볼 수 있다고 생각하셔야 합니다. 그래서 학생 이름이나 기록은 넣지 않고, 누구에게 보여도 괜찮은 연습 화면만 올립니다."

Q8. 그림은 어디서 구하나요? 저작권은요?

- 알아 둘 것: 인터넷 검색 이미지는 대부분 저작권이 있다. AI로 만든 그림도 서비스 이용 조건을 확인한다. 공개 주소에 올리는 자료에는 사용 허락이 분명한 그림만 쓴다.
- 이렇게 답한다: "직접 찍거나 그린 그림, AI로 만든 그림, 사용 허락이 분명한 자료를 쓰세요. 검색해서 나온 그림은 공개 주소에 올리지 않는 게 안전합니다. 처음에는 AI에게 간단한 아이콘까지 함께 그려 달라고 해도 충분합니다."

만들고 고치기

Q9. 코드를 모르는데 AI가 맞게 만들었는지 어떻게 아나요?

- 알아 둘 것: 검토(읽어 보고 판단)와 검증(직접 해 보고 확인)을 나눈다. 완료 기준을 먼저 준다(1부 32쪽, 2부 23쪽).
- 이렇게 답한다: "코드를 읽지 않고 직접 눌러 보고 확인합니다. 2부 23쪽의 5분 확인이 그 방법입니다. 요청할 때 '음료를 누르면 테두리가 두꺼워진다'처럼 완료 기준을 먼저 주면, 그 기준으로 됐는지만 보면 됩니다. 코드가 걱정되면 새 대화에 DESIGN.md와 최신 HTML 파일을 함께 주고 검토를 맡기세요."

Q10. 지침을 넣었는데 AI가 자꾸 안 지켜요.

- 알아 둘 것: 규칙 파일은 길수록 덜 지킨다 [2][5]. 대화가 길어지면 앞 내용이 흐려진다.
- 이렇게 답한다: "지침이 길다면 줄여 보세요. 꼭 지켜야 할 것 몇 줄만 남기는 편이 잘 지킵니다. 같은 부탁을 두 번 했다면 그 한 줄을 지침에 더하고, 대화가 길어졌다면 HANDOFF.md로 정리해 새 대화에서 이어 가세요."

Q11. 사용량 제한에 걸리거나 대화가 끊겼어요.

- 알아 둘 것: 무료 요금제는 사용량 제한이 있다. 인계 파일이 있으면 새 대화나 다른 도구에서도 이어 갈 수 있다.

- 이렇게 답한다: "그래서 인계 파일을 씁니다. 끊기기 전에 'HANDOFF.md로 정리해 줘'라고 해 두면, 다음 날이나 다른 도구에서도 그 파일과 규칙을 붙여 넣고 이어 가시면 됩니다."

Q12. 스킬이나 흑도 꼭 만들어야 하나요? 워크플로와 스킬은 뭐가 다른가요?

- 알아 둘 것: 1부 29쪽 표. '워크플로'는 일의 순서라는 보통 뜻과 도구 기능 이름, 두 뜻으로 쓰인다. 요청문을 저장해 두고 부르던 기능(Codex 커스텀 프롬프트, Claude Code 커맨드, Antigravity 워크플로)은 스킬로 합쳐지는 중이다. Gemini의 Gem도 개인 계정에서는 2026년 11월부터 스킬로 바뀐다 [20][22]. 흑은 정해진 순간에 반드시 실행되고 AI가 고르지 않는다 [24].
- 이렇게 답한다: "시작은 규칙과 개요의 짧은 초안이면 됩니다. 웹 화면을 만들면서 DESIGN.md와 HANDOFF.md를 더하고 실제로 정한 내용을 보완합니다. 같은 점검을 여러 번 하게 되면 그때 스킬로 만들고, 혹은 나중에 필요할 때 보셔도 됩니다. 워크플로는 일의 순서이고, 그 순서를 담아 두는 곳이 스킬입니다. 예전에 '커맨드'나 '워크플로'라고 부르던 저장 기능이 지금은 스킬로 모이고 있어서, 새로 만드실 때는 스킬로 만드시면 됩니다."

Q13. AI가 틀린 내용을 넣으면요?

- 알아 둘 것: AI는 그럴듯하게 틀린 내용을 만들 수 있다. 실제 생활 장면(요금, 메뉴, 절차)은 지역과 시기에 따라 다르다.
- 이렇게 답한다: "안내 문구, 가격, 순서는 선생님이 꼭 확인해 주세요. 특히 버스 요금이나 카페 메뉴처럼 실제 생활 장면은 우리 지역 기준으로 고치셔야 학생이 배운 대로 밖에서 써먹을 수 있습니다."

교실에서 쓰기

Q14. 태블릿에서 소리가 안 나요.

- 알아 둘 것: 태블릿·휴대폰 브라우저는 화면을 한 번 누르기 전에는 소리가 저절로 나지 않게 막는 경우가 많다. 기기의 무음 설정도 확인한다.
- 이렇게 답한다: "첫 화면에 '시작' 버튼을 두고, 누른 뒤부터 소리가 나게 해 달라고 요청하세요. '효과음은 버튼을 누를 때만 재생해 줘'라고 하면 됩니다. 기기가 무음으로 되어 있지 않은지도 함께 봐 주세요."

Q15. 인터넷이 안 되는 교실에서도 쓸 수 있나요?

- 알아 둘 것: HTML 파일 하나로 저장하면 PC에서는 인터넷 없이 열 수 있다. 그림·글꼴을 인터넷에서 불러오게 만들면 인터넷 없이는 보이지 않는다. 태블릿은 기기마다 파일로 여는 방법이 다르다.
- 이렇게 답한다: "PC라면 파일 하나로 저장해 두면 인터넷 없이도 열립니다. 만들 때 '인터넷에서 불러오는 그림이나 글꼴 없이 파일 하나로 만들어 줘'라고 덧붙이세요. 태블릿은 기기마다 파일 여는 방법이 달라서, 인터넷이 되는 곳에서는 주소로 여는 편이 편합니다."

Q16. 만든 자료를 다른 선생님과 나누려면요?

- 알아 둘 것: 공개 주소를 나누면 바로 쓸 수 있다. 고쳐 쓰려면 규칙·개요·화면 기준·인계 문서가 함께 있어야 한다(2부 29쪽).
- 이렇게 답한다: "그대로 쓰실 분께는 주소를, 고쳐 쓰실 분께는 폴더째 드리세요. 규칙과 개요가 같이 있어야 받은 선생님의 AI도 같은 기준으로 이어서 고칩니다."